A Project Report on

Smart Education: Prediction of Learning Effectiveness using Machine Learning

Submitted in partial fulfillment for award of

Bachelor of Technology

Degree

in

Computer Science and Engineering

By

A.Durga Prasad (Y22ACS417) **G.Sushma (Y22ACS457)**

Ch.Venkat Charan (Y22ACS429) **A.Lalitha Jones (Y22ACS413)**

Under the guidance of

S. N. Chandra Sekhar, M.Tech

Assistant Professor

Department of Computer Science and Engineering

Bapatla Engineering College

(Autonomous)

(Affiliated to Acharya Nagarjuna University)

BAPATLA – 522 102, Andhra Pradesh, INDIA

2025-2026

Department of
Computer Science and Engineering

CERTIFICATE

This is to certify that the project report entitled **Smart Education: Prediction of learning Effectiveness using Machine Learning** that is being submitted by A.DURGA PRASAD (Y22ACS417), G.SUSHMA (Y22ACS457), CH.VENKAT CHARAN (Y22ACS429), A.LALITHA JONES (Y22ACS413) in partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science and Engineering to the Acharya Nagarjuna University is a record of bonafide work carried out by them under our guidance and supervision.

Date:

Signature of the Guide
S.N.Chandra Sekhar
Assistant Professor

Signature of the HOD
Dr. M. Rajesh Babu
Assoc. Prof. & Head

DECLARATION

We declare that this project work is composed by ourselves, that the work contained herein is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

A.Durga Prasad (Y22ACS417)

G.Sushma (Y22ACS457)

Ch.Venkat Charan (Y22ACS429)

A.Lalitha Jones (Y22ACS413)

Acknowledgement

We sincerely thank the following distinguished personalities who have offered their valuable advice and support for the successful completion of this work.

We are deeply indebted to our most respected guide, **S. N. Chandra Sekhar, Assistant Professor**, Department of Computer Science and Engineering, for his valuable guidance, constant encouragement, constructive suggestions, and inspiring support throughout the course of this project.

We extend our sincere thanks to **Dr. M. Rajesh Babu, Associate Professor & Head of the Department**, Department of Computer Science and Engineering, for his cooperation and for providing the necessary facilities and resources to carry out this work successfully.

We would also like to express our heartfelt gratitude to our beloved **Principal, Dr. B. Srinivasarao**, for providing the required infrastructure, online resources, and institutional support to complete this project.

We express our sincere thanks to our **Project Coordinators, Dr. C. Prakasa Rao, Professor, Department of CSE, and Dr. R. Venkata Lakshmi, Assistant Professor, Department of CSE**, for their valuable suggestions and guidance in preparing and presenting this document.

We are always thankful to our **parents and family members** for their continuous support, encouragement, and motivation. Without their support, we would not have been able to complete this project work successfully.

Finally, we extend our sincere gratitude to all the teaching faculty and non-teaching staff of the Department of Computer Science and Engineering who helped us directly or indirectly with their cooperation and encouragement.

A.Durga Prasad (Y22ACS417)

G.Sushma (Y22ACS457)

Ch.Venkat Charan (Y22ACS429)

A.Lalitha Jones (Y22ACS413)

ABSTRACT

Information literacy is a foundational skill for college students, essential for adapting to societal demands, engaging in self-directed learning, and fostering lifelong growth. As the need for information-savvy professionals rises, understanding how students acquire these skills and predicting their learning outcomes can greatly enhance educational quality. This project seeks to analyze diverse learning behaviors among college students and predict learning outcomes using advanced machine learning models, ultimately aiming to strengthen information literacy education. Building on prior research that applied supervised classification algorithms (e.g., Decision Tree, K-Nearest Neighbors, Naive Bayes , Neural Networks, and Random Forest), this study introduces Gradient Boosting and classification algorithms (e.g., SVM, AdaBoost). Clustering groups students based on learning behavior patterns, uncovering insights that allow for more targeted interventions, while Gradient Boosting predicts information literacy outcomes with greater accuracy.

Our findings will guide data-driven decision-making in information literacy instruction, helping to optimize teaching methods and resources. By advancing the understanding of how students develop information literacy, this research promotes sustainable educational practices that prepare college students for lifelong learning and innovation in an evolving information-driven world. This refined version improves flow and conciseness, emphasizing the study's goals, methodology, and potential impact.

TABLE OF CONTENTS

CONTENTS **PAGE NO.**

Abstract v

List of Tables ix

List of Figures x

List of Abbreviations xi

CHAPTER 1 – INTRODUCTION 1

1.1 Motivation 2

1.2 Problem Definition 2

1.3 Objective of project 4

CHAPTER 2 – LITERATURE SURVEY 5

2.1 Paper 1 5

2.2 Paper 2: 6

2.3 Paper 3: 7

2.4 Paper 4: 8

2.5 Paper 5: 9

CHAPTER 3 – SYSTEM ANALYSIS 10

3.1 Existing System 10

3.2 Disadvantages of Existing System 11

3.3 Proposed System 12

3.4 Advantages of Proposed System 12

CHAPTER 4 – SYSTEM REQUIREMENTS 14

4.1 Hardware Requirements 14

4.2 Software Requirements 15

4.3 Libraries Used 16

4.4 Functional Requirements 17

4.5 Non-Functional Requirements 18

CHAPTER 5 – SYSTEM DESIGN 19

5.1 System Architecture for proposed system 19

5.2 Database Design 21

5.3 UML Diagrams 21

5.3.1 Use Case Diagram 22

5.3.2 Class Diagram 23

5.3.3 Sequence Diagram 24

5.3.4 Activity Diagram 25

CHAPTER 6 – IMPLEMENTATION 26

6.1 Dataset Description 26

6.2 Pre-Processing of dataset 27

6.3 Algorithms Used 28

6.3.1 XGBoost (Extreme Gradient Boosting) 28

6.3.2 Support Vector Machine (SVM) 29

6.3.3 AdaBoost (Adaptive Boosting) 30

6.4 Feature Extraction 31

6.5 Source Code 32

CHAPTER 7 – TESTING & VALIDATION 36

7.1 Testing Techniques 36

7.1.1 Data Validation Techniques 36

7.1.2 Model Evaluation Techniques 36

7.1.3 Hyperparameter Tuning 37

7.1.4 Performance and Stability Testing 37

7.1.5 User Interface Testing 38

7.2 Test Cases 38

7.2.1 Data Validation Test Cases 38

7.2.2 Predictive Model Evaluation Test Cases 39

CHAPTER 8 – RESULTS.....40

8.1 Dataset Screenshot.....40

8.2 Application Screenshots.....40

CONCLUSION 47

FUTURE ENHANCEMENT 48

REFERENCES.....49

LIST OF TABLES

TABLE NO. **PAGE NO.**

Table 3.1 – Prediction Results of the Classification Model (Existing System).....11

Table 4.1 – Hardware Requirements 14

Table 4.2 – Software Requirements.....16

Table 7.1 – Data Validation Test Cases.....38

Table 7.2 – Predictive Model Evaluation Test Cases 39

LIST OF FIGURES

FIGURE NO. **PAGE NO.**

Fig. 5.1 – System Architecture for Prediction of Student Performance 19

Fig. 5.2 – Use Case Diagram for User Interface.....22

Fig. 5.3 – Class Diagram.....23

Fig. 5.4 – Sequence Diagram.....24

Fig. 5.5 – Activity Diagram 25

Fig. 8.1 – Dataset Features.....40

Fig. 8.2 – Home Page.....41

Fig. 8.3 – Signup Page 41

Fig. 8.4 – Login Page.....42

Fig. 8.5 – Student Features Input (Part 1).....42

Fig. 8.6 – Student Features Input (Part 2).....43

Fig. 8.7 – Student Features Input (Part 3).....43

Fig. 8.8 – Prediction Result 44

Fig. 8.9 – Suggestions Based on Student Level.....44

Fig. 8.10 – Suggestions to Improve Weak Skill Area.....45

Fig. 8.11 – Student Dashboard Showing Faculty Shared Resources.....45

Fig. 8.12 – Faculty Dashboard – Upload Resources.....46

Fig. 8.13 – Faculty Dashboard – Manage Resources.....46

LIST OF ABBREVIATIONS

ML	-	Machine Learning
SVM	-	Support Vector Machine
XGBoost	-	Extreme Gradient Boosting
AdaBoost	-	Adaptive Boosting
KNN	-	K-Nearest Neighbors
NB	-	Naïve Bayes
NN	-	Neural Networks
RF	-	Random Forest
UI	-	User Interface

CHAPTER – 1

INTRODUCTION

Information literacy is a foundational skill for college students, essential for adapting to societal demands, engaging in self-directed learning, and fostering lifelong growth. In the 21st century, students are confronted with an ever-expanding volume of data, making the ability to identify, locate, evaluate, and effectively use information more important than ever. Higher education institutions have increasingly recognized the need to cultivate these skills to prepare graduates for the complex demands of professional and civic life.

As the digital transformation reshapes every domain, information literacy has evolved beyond the ability to find books in a library. Today it encompasses digital literacy, critical evaluation of online resources, ethical use of data, and the capacity to communicate information effectively. Universities and colleges must therefore adopt innovative strategies including the use of data analytics and machine learning to monitor, assess, and improve student information literacy outcomes.

This project proposes a machine learning based system to analyze diverse learning behaviors among college students and predict their information literacy outcomes. The system employs advanced ensemble and classification algorithms, including XGBoost, Support Vector Machine (SVM), and AdaBoost, to achieve higher prediction accuracy than traditional methods. In addition to prediction, the system identifies weak skill areas and delivers personalized recommendations, while a faculty module enables educators to upload and manage study resources for continuous student support.

1.1 MOTIVATION

Traditional approaches to information literacy education are largely generic and do not account for the diversity of student learning behaviors. Educators often lack actionable, data-driven insights about individual students' strengths and weaknesses, making targeted intervention difficult. Prior research (Shi et al., 2023) demonstrated that machine learning algorithms can predict information literacy outcomes from behavioral data, with Random Forest achieving 92.5% accuracy. However, these systems stop at prediction and do not provide personalized improvement plans or faculty–student interaction channels.

This gap between prediction and actionable support motivates the development of an enhanced system that not only classifies students as Poor, Average, or Excellent, but also pinpoints specific areas of weakness and connects students with relevant learning resources uploaded by faculty. By integrating modern ensemble learning techniques with a user-friendly Streamlit interface and a SQLite-backed resource management module, this project bridges the gap between prediction and practice.

1.2 PROBLEM DEFINITION

In an increasingly information-driven society, information literacy is essential for academic success and lifelong learning. However, many college students struggle to develop these skills effectively, leading to poor comprehension, analysis, and utilization of information. Traditional teaching methods lack the personalization required to address individual learning behavior patterns, resulting in ineffective and undifferentiated learning outcomes.

Although previous studies have applied machine learning techniques such as Decision Tree, K-Nearest Neighbors, Naïve Bayes, Neural Networks, and Random Forest to predict information literacy outcomes, several limitations persist. Most existing systems focus solely on prediction and do not provide mechanisms to identify weak skill areas or offer personalized improvement suggestions. Furthermore, there is limited interaction between students and faculty for sharing learning resources, which impedes continuous and self-directed learning.

The key problems addressed by this project are:

1. Existing machine learning models achieve limited accuracy due to the absence of advanced ensemble and boosting techniques.
2. There is no mechanism to identify specific areas where a student's information literacy is weak.
3. Students receive no personalized feedback or recommendations for improving their skills.
4. There is insufficient support infrastructure for faculty to upload and share learning materials with students.
5. The lack of a unified, web-based platform makes it difficult for students to access predictions, suggestions, and resources in one place.

To address these issues, this project implements advanced machine learning algorithms XGBoost, SVM, and AdaBoost to improve prediction accuracy. The system identifies weak skill areas and provides personalized suggestions for improvement. Additionally, a faculty module is introduced to facilitate the uploading and management of study materials, thereby enabling students to access relevant resources and strengthen their learning journey. This comprehensive approach aims to develop a more effective, personalized, and interactive information literacy learning system.

1.3 OBJECTIVE OF PROJECT

The primary objectives of this project are:

1. To analyze the learning behavior patterns of college students using a structured dataset of information literacy attributes.
2. To implement and compare machine learning algorithms XGBoost, SVM, and AdaBoost for accurate prediction of student information literacy outcomes.
3. To identify the key features (behavioral indicators) that most strongly influence a student's information literacy classification.
4. To develop a web-based application using Streamlit that allows students to input their behavioral data and receive instant predictions and personalized improvement suggestions.
5. To identify specific weak skill areas for each student and provide targeted recommendations to enhance their information literacy.
6. To provide a faculty module that enables educators to upload, manage, and share learning resources with students.
7. To evaluate the performance of each algorithm using standard metrics such as accuracy, precision, recall, and F1-score, and to select the optimal model for deployment.

CHAPTER – 2

LITERATURE SURVEY

A comprehensive review of related literature was conducted to understand the state of the art in information literacy prediction, machine learning in education, and digital learning analytics. The following section summarizes five key research papers that form the foundation of this project.

2.1 PAPER 1

Analysis of Learning Behaviour Characteristics and Prediction of Learning Effect for Improving College Students' Information Literacy Based on Machine Learning

Authors: Yong Shi, Fang Sun, Hongkun Zuo, Fei Peng

Journal: IEEE Access, 2023

Summary

The paper explores how machine learning can enhance college students' information literacy by analyzing their learning behavior and predicting learning outcomes. Using data from 320 students, the researchers applied five algorithms Decision Tree, KNN, Naïve Bayes, Neural Networks, and Random Forest to evaluate which behaviors most significantly affect learning. Results showed that behaviors related to information thinking and application had the strongest correlations with successful learning outcomes. Among the algorithms tested, Random Forest achieved the highest accuracy of 92.5%, outperforming other models in terms of precision, recall, and F1-score. The study emphasizes the importance of cultivating critical thinking and innovation and proposes targeted instructional interventions to improve information literacy education.

Advantages

The paper effectively uses machine learning to analyze and predict students' information literacy based on real behavioral data from 320 students. It compares multiple algorithms and identifies key learning behaviors linked to better outcomes, supporting targeted teaching interventions.

Disadvantages

The study is limited to one university, reducing generalizability. It uses only five machine learning models, omitting potentially stronger ensemble methods, and does not address external factors such as student motivation or environmental conditions.

2.2 PAPER 2

Research on the Impact of Information Literacy on the Creativity of Foreign Language Teachers in Chinese Universities Under the Background of Big Data

Authors: Zhou Guoxiang, Mambetova Elnura, Wang Guihua

Journal: International Journal of Distance Education Technologies, Vol. 21, Issue 2, 2023

Summary

This study investigates how information literacy impacts the creativity of foreign language teachers in Chinese universities. Using a sample of 205 teachers, the research applies descriptive analysis, correlation, and regression methods. Results indicate that information literacy significantly enhances teacher creativity, with dimensions such as information knowledge, awareness, and ability showing strong positive effects. Notably, information ethics was found to have no significant impact. The study highlights gender differences and recommends improving information literacy through teacher training and institutional policy.

Advantages

The paper offers valuable insights into how information literacy influences teacher creativity in underrepresented regions of China, using solid statistical methods and identifying key literacy dimensions that significantly boost creativity.

Disadvantages

The study is limited by its small, region-specific sample and reliance on self-reported data, which may affect accuracy. It does not consider other influencing factors like institutional support or teaching conditions.

2.3 PAPER 3

Defining Digital Literacy: A Case Study of Australian Universities

Authors: Nona Press, Puvaneswari P. Arumugam, Kevin Ashford-Rowe

Conference: ASCILITE 2019 Conference Proceedings, Singapore University of Social Sciences

Summary

The paper explores how Australian universities conceptualize and implement digital literacy within their institutional frameworks. By analyzing publicly available documents from 42 universities, the study reveals a lack of consensus on the definition of digital literacy, with only 16 institutions offering direct definitions. These definitions range from basic technical skills to broader competencies involving critical thinking, communication, and lifelong learning. The study advocates for strategic planning and inclusive approaches to foster critical digital competencies.

Advantages

The paper provides a timely and detailed look into how Australian universities define and approach digital literacy, broadening the concept beyond technical skills and emphasizing lifelong learning and adaptability.

Disadvantages

The study is limited to publicly available data, missing internal strategies and stakeholder perspectives. It does not provide concrete implementation strategies, and varied use of terms across institutions adds to definitional ambiguity.

2.4 PAPER 4

Information Literacy, Theory and Practice in Education

Authors: Dubravka Mandušić, Lucija Blašković

Journal: Revista Românească pentru Educaţie Multidimensională, Vol. 5, Issue 1, pp. 47–58, 2013

Summary

The paper explores the vital role of information literacy in modern education systems, particularly in the context of globalization, digital transformation, and lifelong learning. It defines information literacy as the ability to identify, locate, evaluate, and effectively use information. The authors argue that information literacy forms the foundation of successful education and professional life. A key example is the Carleton College Mellon Initiative, which successfully embedded information literacy into the curriculum through collaborative departmental efforts.

Advantages

The paper provides a clear framework for understanding information literacy and links it effectively to lifelong learning, promoting critical thinking, active learning, and autonomy among students.

Disadvantages

The paper is mostly theoretical and lacks practical data or case studies. It does not deeply address challenges like limited resources or faculty training, and has a slightly idealistic tone that may weaken its real-world applicability.

2.5 PAPER 5

A Heterogeneity Study on the Effect of Digital Education Technology on the Sustainability of Cognitive Ability for Middle School Students

Authors: Desheng Yan, Guangming Li

Journal: Sustainability, Vol. 15, Article 2784, 2023

Summary

The paper explores how digital education technologies influence the cognitive abilities of middle school students. Drawing on data from the China Education Panel Survey (CEPS), the study uses OLS regression, Coarsened Exact Matching (CEM), and quantile regression to evaluate the impact. The findings suggest that digital education technology has a generally positive effect on cognitive development; however, students with higher initial cognitive ability tend to benefit more, while those with lower ability may experience limited outcomes.

Advantages

The study stands out for its strong methodological approach, using a large dataset and advanced statistical tools to ensure reliable findings. It effectively highlights the varied impact of digital education technology across different student groups.

Disadvantages

The study lacks qualitative perspectives and is limited to a specific national context. It also does not fully address practical challenges such as unequal access to technology or the need for teacher training.

CHAPTER – 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The existing system employs supervised machine learning algorithms to analyze and predict college students' information literacy levels based on their learning behaviors. It utilizes models such as Decision Tree, K-Nearest Neighbors (KNN), Naïve Bayes, Neural Networks, and Random Forest to classify students into different proficiency levels based on critical factors like critical thinking ability, application skills, and overall learning outcomes.

The system processes student learning behavior data—including engagement levels, study habits, and information evaluation skills—to provide insights into their literacy proficiency. A dataset of 320 college students' information literacy learning records was used for model training and evaluation. The Pearson correlation algorithm was applied to identify the most influential learning behaviors, revealing that information thinking and application skills have the strongest correlation with student performance.

Among the models tested, Random Forest emerged as the best performer with an accuracy of 92.50%, outperforming other classification techniques in terms of precision, recall, and F1-score.

3.1.1 Result and Analysis of Existing System

The existing system successfully predicts students' information literacy proficiency. The detailed performance comparison of all models is presented in Table 4.1 below.

Table 3.1: Prediction Results of the Classification Model (Existing System)

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	Kappa Coeff.
Decision Tree	84.17%	76.00%	88.63%	81.83%	0.703
K-Nearest Neighbor (KNN)	90.83%	80.87%	93.96%	86.92%	0.831
Naïve Bayes	90.00%	93.06%	69.97%	79.88%	0.806
Neural Networks	91.67%	60.83%	63.15%	63.11%	0.837
Random Forest	92.50%	84.56%	94.81%	89.39%	0.859

The above results clearly indicate that while Random Forest achieves the highest accuracy (92.50%), integrating advanced boosting techniques and adaptive learning models could further optimize prediction accuracy, improve computational efficiency, and enable real-time student interventions.

3.2 DISADVANTAGES OF EXISTING SYSTEM

1. Limited Accuracy and Performance: Traditional models such as Decision Tree and Naïve Bayes achieve lower accuracy compared to modern ensemble methods.
2. Lack of Boosting Techniques: The absence of algorithms such as XGBoost and AdaBoost prevents further improvements in model accuracy and bias reduction.
3. High Computational Cost: Models such as KNN and Neural Networks require high processing power and memory, making them impractical for large datasets.
4. No Personalized Learning Recommendations: The existing system only provides a classification label and does not identify weak skill areas or suggest improvement strategies.
5. No Faculty Support Module: There is no facility for educators to upload study materials, limiting student access to learning resources.

3.3 PROPOSED SYSTEM

The proposed system enhances the prediction of students' information literacy levels by integrating advanced machine learning techniques XGBoost, Support Vector Machines (SVM), and AdaBoost. These models improve accuracy by reducing bias and variance while refining weak classifiers. Unlike the existing system, which relied on traditional models, the proposed system offers better classification and predictive performance by identifying key learning behavior patterns and enabling personalized education strategies.

A major improvement is the ability to identify specific weak skill areas for each student and provide targeted improvement suggestions. The system clusters students based on learning behaviors, allowing educators to tailor teaching strategies to different learner groups and identify at-risk students for early intervention.

The proposed system addresses computational inefficiencies by utilizing optimized boosting algorithms, enhancing efficiency and scalability for real-time applications. Additionally, a faculty module is introduced that allows educators to upload, manage, and share study materials and notes with students, supporting continuous learning beyond prediction.

By leveraging predictive analytics and personalized feedback mechanisms, the proposed system fosters continuous learning improvement, preparing students for academic and professional success in an information-driven world.

3.4 ADVANTAGES OF PROPOSED SYSTEM

1. Higher Accuracy and Improved Prediction Performance: Ensemble methods such as XGBoost and AdaBoost significantly improve classification accuracy over traditional models.
2. Personalized Learning and Targeted Interventions: The system identifies individual weak skill areas and provides customized improvement suggestions.
3. Optimized Computational Efficiency: Boosting algorithms are computationally efficient and scalable, suitable for large datasets and real-time deployment.

4. Data-Driven Decision-Making for Educators: Provides actionable insights that help instructors refine curriculum design, adjust teaching methods, and allocate resources effectively.
5. Faculty Resource Module: Enables educators to upload and manage study materials, facilitating resource sharing and continuous student support.
6. Web-Based User Interface: A Streamlit-based application provides easy access for both students and faculty through an intuitive, responsive interface.

CHAPTER – 4

SYSTEM REQUIREMENTS

4.1 HARDWARE REQUIREMENTS

Table 4.2: Hardware Requirements

S.No.	Hardware Component	Minimum Specification
1	Processor	Intel Core i5 (2.0 GHz) or higher
2	RAM	Minimum 8 GB (16 GB recommended)
3	Storage	Minimum 50 GB free disk space (SSD preferred)
4	Operating System	Windows 10 / 11 (64-bit)
5	Display	1280 x 768 resolution or higher

4.1.1 Processor Requirement

The project requires a processor of Intel Core i5 (2.0 GHz) or higher to efficiently handle the complex computations involved in training machine learning models such as XGBoost, SVM, and AdaBoost. A higher processing power ensures faster execution of classification and clustering algorithms, enabling real-time predictions and data analysis. Machine learning tasks, especially those involving large datasets, require multi-core processing capabilities to optimize model training time and enhance overall system performance.

4.1.2 RAM Requirement

A minimum of 8 GB RAM is required to support the efficient execution of machine learning models and data processing tasks. Machine learning algorithms involve handling large datasets, performing multiple iterations, and training complex models, which require sufficient memory to prevent system slowdowns. Higher RAM capacity allows for seamless data preprocessing, feature extraction, and model evaluation.

4.1.3 Operating System Requirement

The system is designed to run on Windows 10 / 11 (64-bit), ensuring compatibility with the necessary machine learning frameworks, development environments, and libraries. Windows OS provides a stable platform for executing Python-based data science and machine learning applications, supporting tools like Anaconda, Jupyter Notebook, and Scikit-Learn.

4.2 SOFTWARE REQUIREMENTS

4.2.1 Python Requirement

Python 3.x serves as the core programming language for implementing the machine learning models in this project. It offers a rich ecosystem of libraries such as Scikit-Learn for model training, Pandas for data manipulation, NumPy for numerical computations, and Matplotlib for data visualization. Python's flexibility and efficiency make it ideal for developing and deploying machine learning algorithms in an educational data science environment.

4.2.2 Visual Studio Code Requirement

Visual Studio Code (VS Code) is a lightweight, open-source code editor developed by Microsoft that supports multiple programming languages. It provides features such as syntax highlighting, code completion, debugging, and version control integration through Git. Extensions enhance its functionality, allowing support for frameworks, languages, and tools including Python, JavaScript, and Docker.

4.2.3 Jupyter Notebook Requirement

The project is developed and tested in Jupyter Notebook, an interactive computing environment that supports real-time coding, debugging, and visualization. Jupyter Notebook enables developers to write, execute, and document code in a structured format, making it easy to track data processing steps and model performance.

Table 4.1: Software Requirements

S.No.	Category	Requirement
1	Programming Language	Python 3.x
2	IDE / Editor	Visual Studio Code (VS Code), Jupyter Notebook
3	Framework	Streamlit
4	Database	SQLite
5	Frontend Interface	Streamlit UI, CSS

4.3 LIBRARIES USED

Streamlit

Streamlit is a Python library used to build interactive and visually appealing web applications, especially for data science and machine learning workflows. In this application, it powers the entire frontend, handling everything from layout configuration to user interactions, form inputs, and page navigation. Streamlit's ability to manage session states and easily integrate UI widgets makes it an ideal choice for creating user-friendly machine learning tools.

SQLite3

SQLite3 is Python's built-in module for interacting with SQLite databases. It allows the application to store and retrieve user registration and login data efficiently. The application uses SQLite to maintain a users.db database where information such as usernames, passwords, emails, and demographic data is stored.

Pandas

Pandas is a powerful data analysis and manipulation library used to load the dataset from an Excel file and perform preprocessing tasks such as filling missing values and selecting relevant features. Pandas provides intuitive and flexible data structures like DataFrames, which are central to preparing data for machine learning models.

NumPy

NumPy supports numerical computations in Python and complements Pandas during data preprocessing. It is used to calculate mean values for numeric columns and assist in converting them into appropriate formats. Its efficient array operations make it a foundational tool for any data-driven Python application.

Scikit-Learn (sklearn)

Scikit-Learn provides the core machine learning utilities used in this project, including LabelEncoder for categorical encoding, train_test_split for dataset partitioning, accuracy_score for performance evaluation, SVC (Support Vector Classifier), and AdaBoostClassifier with DecisionTreeClassifier as the base estimator.

XGBoost

The XGBClassifier from the XGBoost library is a highly optimized, scalable gradient boosting machine learning algorithm widely used for structured or tabular data. It is configured with specific parameters for depth, learning rate, and number of estimators to enhance prediction accuracy.

4.4 FUNCTIONAL REQUIREMENTS

The Information Literacy Prediction System is designed to assess and categorize an individual's information literacy level using machine learning algorithms. The key functional requirements include:

1. Input Data: Collect structured input from users via forms including behavioral attributes such as IPC, IAC, LLC, ISK, IAS, IT, IB, IE, and ILR scores.
2. Preprocessing: Clean, normalize, and encode user-provided data for model compatibility, including handling missing values and label encoding.
3. Feature Extraction: Identify and utilize significant attributes influencing information literacy classification.
4. Prediction Models: Implement multiple algorithms (XGBoost, SVM, AdaBoost) for prediction with a user-selectable option.

5. User Interface: A Streamlit-based UI for easy navigation, including Login, Signup, Home, and Prediction pages.
6. Weak Skill Identification: Automatically identify the skill categories where a student scores lowest and flag them for attention.
7. Personalized Suggestions: Provide actionable recommendations tailored to each student's predicted literacy level and weak areas.
8. Faculty Module: Allow faculty members to upload study materials and notes; allow students to access and download these resources.
9. Results Display: Show predicted literacy level, model accuracy, and corresponding improvement suggestions.

4.5 NON-FUNCTIONAL REQUIREMENTS

The system must fulfill the following non-functional requirements to ensure an optimal user experience and dependable performance:

1. Performance: Quick response time for form submission and prediction; high accuracy of models ensured through continuous performance tracking.
2. Availability: System should be operational and accessible with minimal downtime.
3. Scalability: Capable of handling increased user load and data input without performance degradation.
4. Usability: Simple, intuitive interface designed for users with varying technical backgrounds.
5. Maintainability: Clean, modular code structure for easier updates, debugging, and model integration.
6. Compatibility: Functional across different platforms and devices—desktop, mobile, and various browsers.
7. Reliability: Consistent performance and accurate predictions over time.

CHAPTER – 5

SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE FOR PROPOSED SYSTEM

The proposed system is designed to classify student learning behavior using machine learning techniques. It starts with the collection of student data—including academic performance, attendance, and behavioral traits. After preprocessing the data to ensure consistency and accuracy, it is divided into training and testing sets. Classification algorithms such as XGBoost, SVM, and AdaBoost are applied to train models that can predict student performance levels. Based on the model's output, students are categorized into Poor, Average, or Excellent. This system supports educators in identifying at-risk students and promoting targeted intervention to enhance overall learning outcomes.

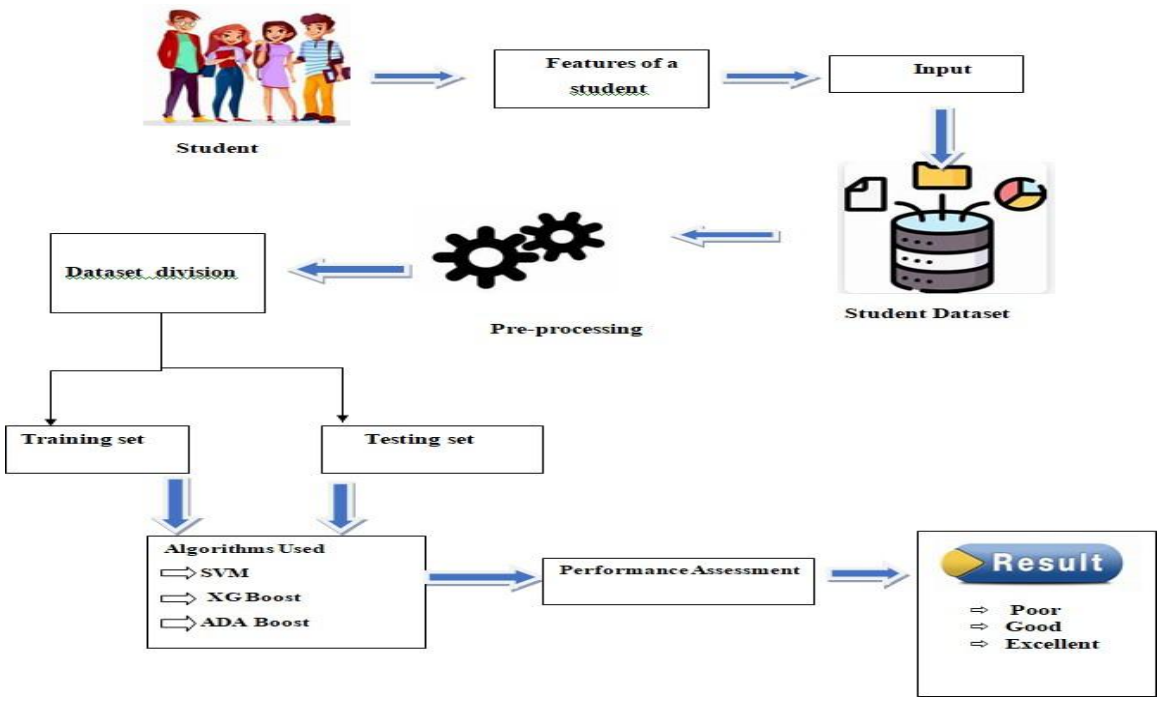

Fig. 5.1: System Architecture for Prediction of Student Performance

Architectural Design

The architectural design is modular, with distinct layers that ensure a smooth data flow. It includes a Data Collection Layer that gathers student data, followed by a Preprocessing Layer to clean and normalize it. The Dataset Division Layer separates the

data into training and testing sets. The Machine Learning Layer applies classification models, and the Performance Assessment Layer evaluates model accuracy. Finally, the Output Layer classifies students into learning behavior categories. Each layer is functionally independent but interconnected, allowing easy updates, scalability, and efficient system performance.

Logical Design

The logical design outlines how the system processes and transforms data into meaningful classifications. It begins with inputting student features, followed by preprocessing steps such as data cleaning and encoding. The dataset is then split into training and testing sets. Machine learning algorithms are applied to the training data, and their predictions are tested and evaluated. The best-performing model is selected to classify students as Poor, Average, or Excellent.

Physical Design

The physical design relates to the actual input and output processes of the system, specifying how data is input, verified, processed, and displayed. The key physical design elements include:

1. Input requirements: Structured form-based data entry via Streamlit UI.
2. Output requirements: Predicted literacy level, model accuracy, and personalized suggestions.
3. Storage requirements: SQLite database (users.db) for user data and uploaded resources.
4. Processing requirements: Python-based ML pipeline with Scikit-Learn and XGBoost.
5. System control and recovery: Session state management via Streamlit.

5.2 DATABASE DESIGN

A database is an organized collection of data that helps in storing and managing information efficiently. In this project, SQLite is used as the database management system due to its simplicity, portability, and easy integration with Python-based applications. The database is designed to support both student and faculty functionalities.

The system consists of two primary tables:

- Users Table: Stores information about all registered users, including students and faculty members. Fields include: User ID (Primary Key), Full Name, Username (Unique), Password (Hashed), Email, Gender, Age, and Role (Student/Faculty).
- Resources Table: Stores details of study materials uploaded by faculty members. Fields include: Resource ID (Primary Key), Filename, Original File Name, Description, Uploader (Foreign Key to Users), and Upload Date.

Proper constraints such as primary keys and unique fields are used to maintain data integrity. The role attribute in the Users table enables role-based access control within the system.

5.3 UML DIAGRAMS

UML (Unified Modeling Language) is a visual modeling language used to make software blueprints, enabling visualization, specification, construction, and documentation of software systems.

The following UML diagrams have been used to model this system:

5.3.1 Use Case Diagram

A use case diagram describes a sequence of actions that provide measurable value to an actor. An actor is a person, organization, or external system that plays a role in one or more interactions with the system. The use case diagram below illustrates the interactions between Student and Faculty actors with the system, including login/signup, data input, prediction, viewing suggestions, and resource management.

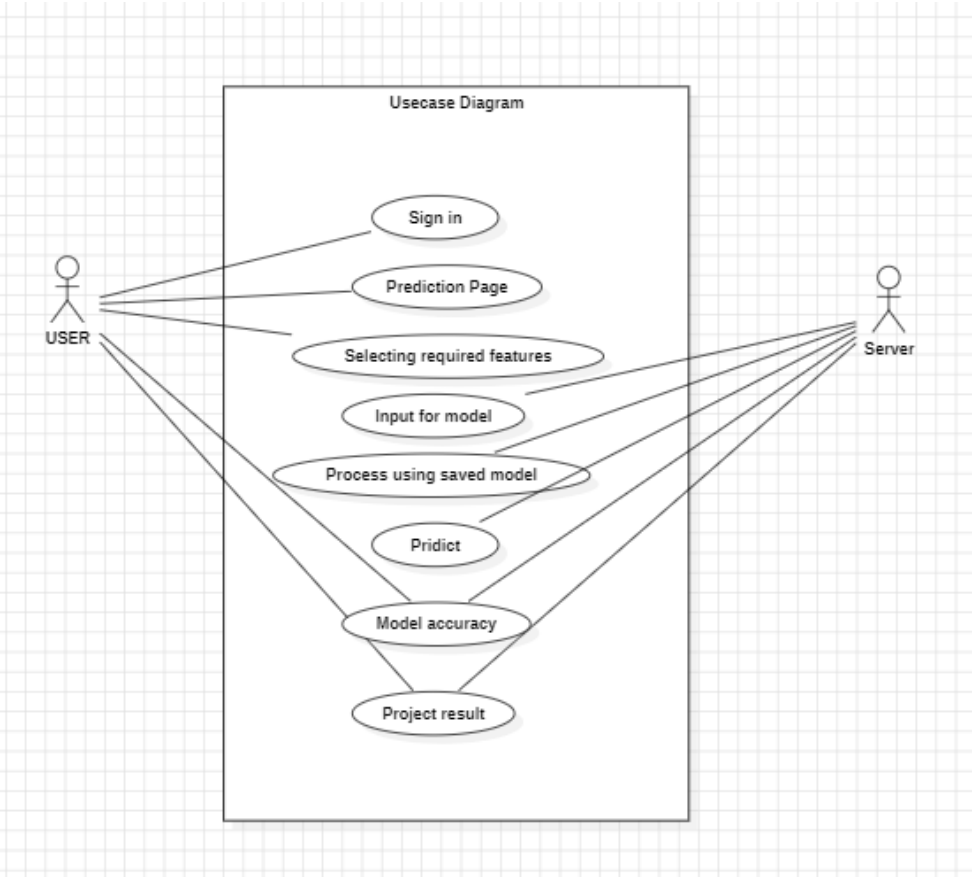

Fig. 5.2: Use Case Diagram for User Interface

5.3.2 Class Diagram

A class diagram describes the static structure of the system. It is a graphic representation of the static view that shows a collection of declarative model elements such as classes, types, and their contents and relationships. The class diagram below shows the key classes—User, Student, Faculty, Predictor, ResourceManager—and their associations.

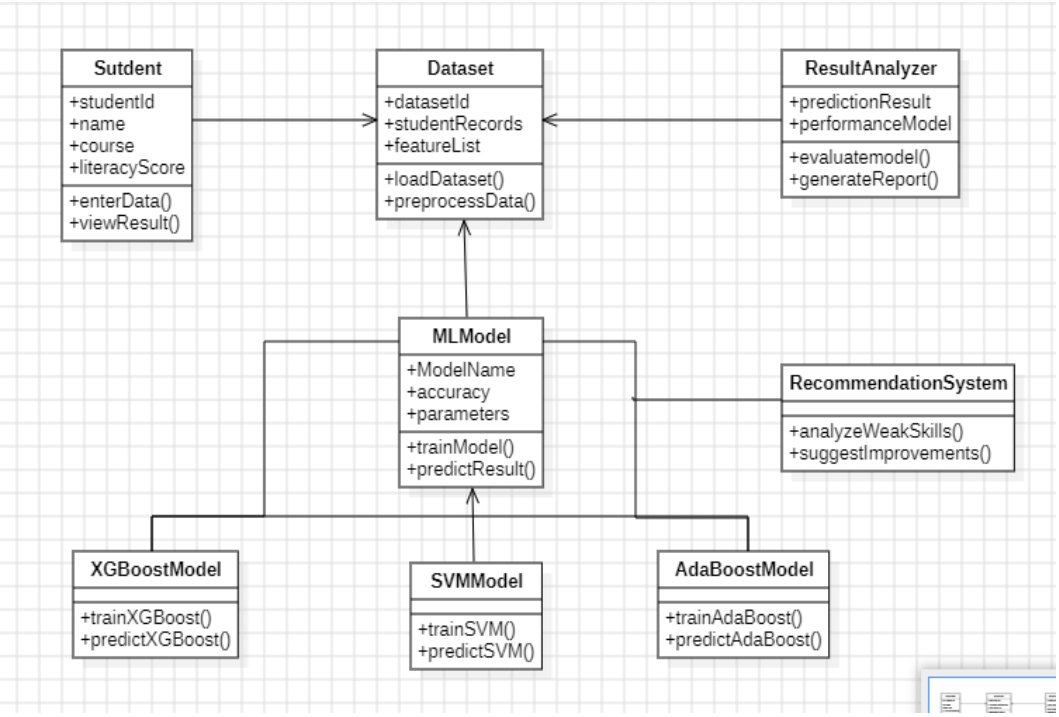

Fig. 5.3: Class Diagram

5.3.3 Sequence Diagram

UML sequence diagrams model the flow of logic within a system in a visual manner, enabling documentation and validation of logic. Sequence diagrams are the most popular UML artifacts for dynamic modeling, focusing on identifying the behavior within the system. The diagram below shows the flow from user login through feature input, model selection, prediction, and output display.

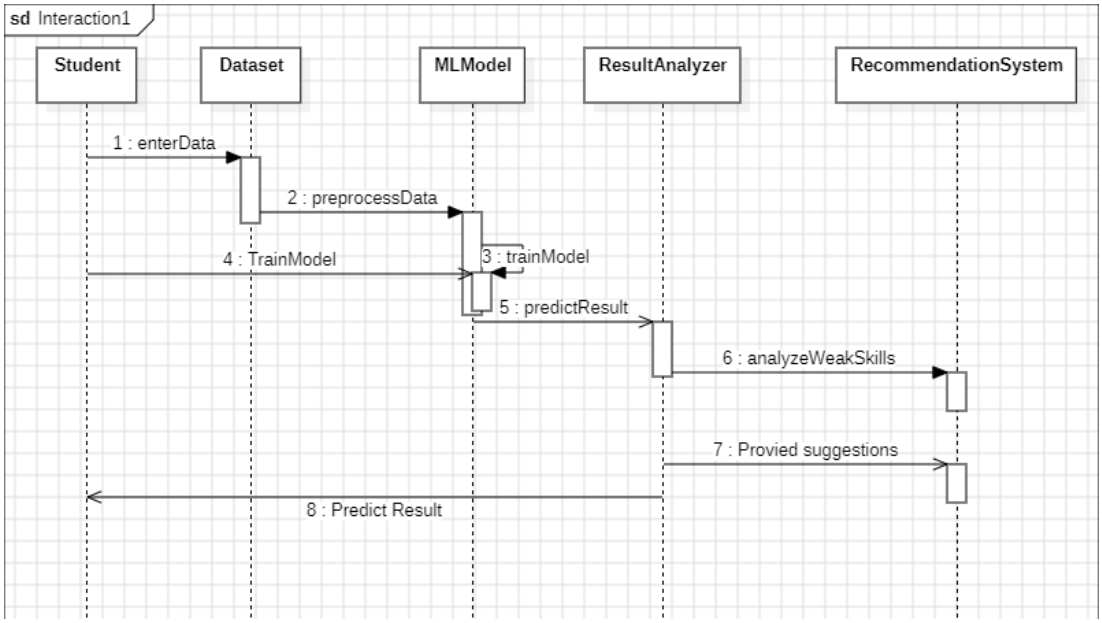

Fig. 5.4: Sequence Diagram

5.3.4 Activity Diagram

Activity diagrams document workflows in a system, from the business level down to the operational level. The activity diagram below illustrates the complete workflow of the system, including user authentication, data entry, preprocessing, model prediction, result display, and faculty resource management.

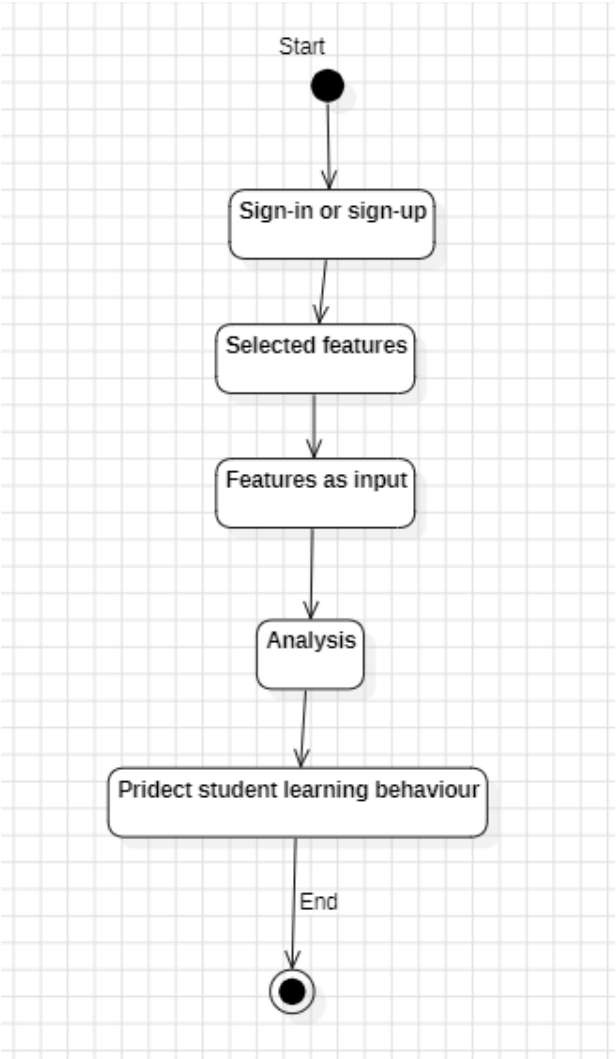

Fig. 5.5: Activity Diagram

CHAPTER – 6

IMPLEMENTATION

6.1 DATASET DESCRIPTION

The dataset contains responses and scores from college students on various indicators related to their learning behavior. Each student has been evaluated using a series of metrics grouped under psychological and behavioral attributes. These indicators are measured on a numerical scale and reflect individual aspects of students' learning processes, such as information processing, learning control, and information seeking. Additional demographic details including gender and educational qualification are also included, along with a final classification of each student's learning performance labeled as Poor, Average, or Excellent.

The dataset is well-structured and suitable for classification problems in machine learning. The presence of diverse behavioral dimensions along with a labeled outcome enables effective training of models for predicting student performance.

Feature Explanation

IPC1, IPC2, IPC3 – Information Processing Capacity: These features assess a student's ability to absorb, organize, and mentally process new information, including how quickly they grasp new concepts and relate them to prior knowledge.

IAC1, IAC2 – Information Acquisition Capability: These features measure the effectiveness with which students gather knowledge from various sources like textbooks, lectures, or digital content. Higher scores suggest proactive learning strategies.

LLC1, LLC2 – Learning and Logical Control: These indicators reflect a student's ability to regulate their learning habits and apply logical thinking, including planning, managing, and controlling study routines.

ISK1–ISK4 – Information Seeking Knowledge: This group evaluates a student's behavior in actively seeking relevant information, including internet searches, consulting peers or teachers, and accessing external resources.

IAS1–IAS4 – Information Analysis Skills: These features gauge how well students can critically analyze information, including identifying key points, comparing perspectives, evaluating arguments, and breaking down complex ideas.

IT1–IT4 – Information Transformation: These metrics reflect the student's ability to apply learned knowledge in new ways—using concepts in real-life scenarios, solving unfamiliar problems, or synthesizing ideas from multiple sources.

IB1–IB4 – Information Behavior: These features focus on how students interact with information behaviourally whether they are consistent, organized, or strategic in handling data throughout the learning process.

IE1–IE3 – Information Evaluation: This set examines how effectively students judge the quality and usefulness of information, including distinguishing credible sources and assessing the relevance of data.

ILR1, ILR2 – Information and Learning Reflection: These features cover the metacognitive aspect how students reflect on their learning experiences, adjust strategies, and engage in self-regulated learning.

GENDER: Indicates the gender of the student (Male/Female).

EDUCATIONAL QUALIFICATION: Specifies the academic level of the student, such as UG (Undergraduate) or Ph.D.

LABEL: This is the target variable the final classification of the student's learning performance: Poor, Average, or Excellent.

6.2 PRE-PROCESSING OF DATASET

Preprocessing is a crucial step in analyzing learning behavior and applying predictive analytics. It ensures the dataset is clean, structured, and ready for modeling.

6.2.1 Data Cleaning

Handling Missing Values: Missing data in numerical columns is addressed using mean imputation (fillna with column mean). For categorical columns such as GENDER and EDUCATIONAL QUALIFICATION, mode imputation is applied. Rows with excessive missing values are removed to maintain data quality.

6.2.2 Data Transformation

Feature Engineering involves creating new features that provide additional insights. Useful engineered features include Engagement Score (combining library visits, articles accessed, and time spent on research), Citation Accuracy (ratio of cited sources to total references), and Search Efficiency (number of queries before selecting a relevant resource).

Categorical features—GENDER, EDUCATIONAL QUALIFICATION, and LABEL are encoded into numerical values using Scikit-Learn's LabelEncoder, making them compatible with machine learning models.

6.2.3 Data Normalization

Data Normalization ensures consistency across features, particularly important for distance-based algorithms such as SVM. Two techniques are applied: Standardization (Z-score normalization), which scales features to have zero mean and unit variance, and Min-Max Scaling, which maps values to a [0, 1] range.

6.2.4 Splitting Data for Training and Testing

The dataset is divided into training and testing sets using an 80:20 split ratio. The training set (80%) is used to train the models, while the testing set (20%) is reserved for evaluation. This approach prevents overfitting and ensures the model generalizes well to unseen data. Stratified splitting is applied to handle any class imbalance in the dataset.

6.3 ALGORITHMS USED

In this project, three machine learning algorithms have been implemented and compared for predicting student information literacy outcomes:

6.3.1 XGBoost (Extreme Gradient Boosting)

XGBoost (Extreme Gradient Boosting) is a machine learning algorithm that enhances gradient boosting with efficiency, speed, and accuracy. It is widely used for classification, regression, and ranking problems due to its strong predictive performance and ability to handle large datasets. XGBoost is built on the Gradient Boosting Decision Tree (GBDT) model, where decision trees are built sequentially and each tree corrects the errors of the previous ones using gradient descent on a loss function.

Key Components of XGBoost

- Boosting Framework: Sequential decision trees, each correcting errors of previous trees.
- Second-Order Taylor Expansion: Improves accuracy over standard gradient boosting.
- Built-in L1 (Lasso) and L2 (Ridge) Regularization: Prevents overfitting.
- Parallel and Distributed Computing: Multi-threading during tree building for faster training.
- Pruning and Early Stopping: Avoids unnecessary complexity.

Advantages of XGBoost

- High Performance and Accuracy: Second-order optimization improves predictive power.
- Speed and Efficiency: Uses parallel computing and optimized memory with compressed block structure.
- Handles Missing Values: Has a built-in mechanism to handle sparse data.
- Feature Importance: Provides feature importance scores for model interpretability.

6.3.2 Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised learning algorithm used for classification and regression tasks. It finds the optimal decision boundary (hyperplane) that best separates data points into different classes by maximizing the margin between the closest points called support vectors of each class. SVM is effective in high-dimensional spaces and works well even with non-linearly separable data by using kernel functions to transform data into higher dimensions.

Key Concepts of SVM

- Hyperplane: The optimal decision boundary that maximizes the margin between classes.
- Support Vectors: The data points closest to the hyperplane that determine its position.
- Kernel Trick: Transforms data into higher-dimensional spaces using kernels (Linear, RBF, Polynomial).
- Regularization Parameter (C): Controls the trade-off between margin maximization and misclassification.

Advantages of SVM

- Effective for High-Dimensional Data: Works well even when the number of features exceeds the number of samples.
- Robust to Overfitting: Soft margin SVM prevents overfitting by allowing some misclassifications.
- Effective for Linearly and Non-Linearly Separable Data: Kernel trick enables flexible boundary detection.

6.3.3 AdaBoost (Adaptive Boosting)

Adaptive Boosting (AdaBoost) is a boosting ensemble algorithm that combines multiple weak learners typically decision stumps (one-level decision trees) to create a strong classifier. It assigns weights to each data point, focusing more on misclassified instances in subsequent iterations. Each weak classifier is assigned a weight based on its accuracy, and classifiers with lower error receive higher weight, contributing more to the final model.

Key Components of AdaBoost

- Weak Learners: Decision stumps that perform slightly better than random guessing.
- Sample Weighting: Misclassified instances receive higher weights in each iteration.
- Classifier Weighting: Each classifier's contribution is proportional to its accuracy.

- Iterative Training: Runs for T iterations, each adding a new corrective weak learner.

Advantages of AdaBoost

- Improves Weak Learners: Converts multiple weak classifiers into a strong classifier.
- High Accuracy: Often outperforms single classifiers and other ensemble methods.
- Feature Selection Ability: Assigns higher importance to relevant features, aiding dimensionality reduction.
- Less Prone to Overfitting: Compared to individual decision trees, AdaBoost generalizes better.

6.4 FEATURE EXTRACTION

Feature extraction plays a crucial role in analyzing students' learning behavior and applying predictive analytics to improve information literacy. The goal is to extract meaningful patterns from student interactions, behaviors, and performance data to design better educational interventions.

The feature extraction process includes the following steps:

1. Selection of Relevant Behavioral Attributes: All behavioral feature columns (IPC1–IPC3, IAC1–IAC2, LLC1–LLC2, ISK1–ISK4, IAS1–IAS4, IT1–IT4, IB1–IB4, IE1–IE3, ILR1–ILR2, GENDER, EDUCATIONAL QUALIFICATION) are selected as input features.
2. Correlation Analysis: Pearson correlation is applied to identify features most strongly correlated with the LABEL (target variable), confirming that information thinking and application skills are the most influential predictors.
3. Removal of Redundant Features: Features with very low correlation to the target variable or high inter-feature correlation are identified and may be removed to reduce dimensionality.
4. Label Encoding of Categorical Features: GENDER and EDUCATIONAL QUALIFICATION are encoded into numerical values using LabelEncoder.

5. Final Feature Vector: The final feature set comprises all encoded behavioral attributes used as input X, with LABEL as the output y.

6.5 SOURCE CODE

The following section presents the core source code of the Information Literacy Prediction System. The application is implemented in Python using Streamlit for the user interface, Scikit-Learn for machine learning, and XGBoost for gradient boosting.

```
import re
import os
import streamlit as st
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.svm import SVC
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from xgboost import XGBClassifier
from db import get_all_resources

DATA_PATH = os.path.join(os.path.dirname(__file__), 'DATA.xlsx')
UPLOAD_DIR = os.path.join(os.path.dirname(__file__), 'uploads')
os.makedirs(UPLOAD_DIR, exist_ok=True)

@st.cache_resource(show_spinner='Loading dataset...')
def load_data():
    df = pd.read_excel(DATA_PATH)
    df.columns = df.columns.str.strip()
    if "GENDER" in df.columns: df["GENDER"] = df["GENDER"].astype(str)
```

```
    if "EDUCATIONAL QUALIFICATION" in df.columns: df["EDUCATIONAL
QUALIFICATION"] = df["EDUCATIONAL QUALIFICATION"].astype(str)
    if "LABEL" in df.columns: df["LABEL"] = df["LABEL"].astype(str)

    # Fill numeric missing values
    numeric_cols = df.select_dtypes(include=np.number).columns
    if len(numeric_cols) > 0:
        df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].mean())

    if "GENDER" in df.columns:
        df["GENDER"] = df["GENDER"].replace('nan',
                                np.nan).fillna(df["GENDER"].mode()[0])
    if "EDUCATIONAL QUALIFICATION" in df.columns:
        df["EDUCATIONAL QUALIFICATION"] = df["EDUCATIONAL
QUALIFICATION"].replace('nan', np.nan).fillna(df["EDUCATIONAL
QUALIFICATION"].mode()[0])

    label_encoders = {}
    for col in ["GENDER", "EDUCATIONAL QUALIFICATION", "LABEL"]:
        if col in df.columns:
            le = LabelEncoder()
            df[col] = le.fit_transform(df[col])
            label_encoders[col] = le

    correlation = df.corr(numeric_only=True)["LABEL"].abs()
    selected_features = correlation[correlation > 0.2].index.tolist()

    if "LABEL" in selected_features:
        selected_features.remove("LABEL")
    if not selected_features:
        selected_features = [c for c in df.select_dtypes(include="number")
```

```
                                        .columns if c !="LABEL"]
    X = df[selected_features]
    y = df["LABEL"]

    feature_means = {f: float(X[f].mean()) for f in selected_features}
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
     return X_train, X_test, y_train, y_test, selected_features,
                                    label_encoders feature_means
@st.cache_resource(show_spinner="Training models…")
def train_models(_X_train, _y_train, _X_test, _y_test):
    """Trains XGBoost, SVM, and AdaBoost classifiers."""
    models = {}

    xgb = XGBClassifier(use_label_encoder=False, eval_metric="mlogloss",
                    n_estimators=50,learning_rate=0.2,max_depth=4)
    xgb.fit(_X_train, _y_train)
    xgb_acc = accuracy_score(_y_test, xgb.predict(_X_test)) * 100
    models['XGBoost'] = (xgb, xgb_acc)

    svm = SVC(kernel='linear', C=1.0, random_state=42)
    svm.fit(_X_train, _y_train)
    svm_acc = accuracy_score(_y_test, svm.predict(_X_test)) * 100
    models['SVM'] = (svm, svm_acc)

    ada = AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=2),
            n_estimators=100,learning_rate=0.1,random_state=42)
    ada.fit(_X_train, _y_train)
    ada_acc = accuracy_score(_y_test, ada.predict(_X_test)) * 100
    models['AdaBoost'] = (ada, ada_acc)
    return models
def predict(model, input_df: pd.DataFrame, label_encoder: LabelEncoder):
```

```
    """Run inference on a single-row DataFrame."""
    prediction = model.predict(input_df)[0]
    return label_encoder.inverse_transform([int(prediction)])[0]

def _clean_feature_name(feature: str) -> str:
    cleaned = re.sub(r'\s*\([^)]*\)\s*$', '', feature).strip()
    if cleaned == cleaned.upper():
        cleaned = cleaned.title()
    return cleaned

def detect_weak_topics(input_data: dict, feature_means: dict) -> list:
    """Identify features where the student scored below the dataset mean."""
    weak = []
    for feature, value in input_data.items():
        if feature in feature_means and isinstance(value, (int, float)):
            mean = feature_means[feature]
            if value < mean:
                weak.append((_clean_feature_name(feature), value, mean, mean - value))
    weak.sort(key=lambda x: x[3], reverse=True)
    return [(name, score, mean) for name, score, mean, _ in weak[:5]]
```

CHAPTER – 7

TESTING & VALIDATION

Testing is a critical phase in the software development lifecycle. It ensures that the system functions correctly, performs efficiently, and produces accurate, reliable outputs. For a machine learning–based system such as this, testing encompasses both functional testing of the application and rigorous validation of the machine learning models.

7.1 TESTING TECHNIQUES

7.1.1 Data Validation Techniques

Data Cleaning and Preprocessing

- Missing Data Analysis: Uses techniques such as mean/mode imputation and KNN imputation or drops rows with excessive missing values.
- Data Consistency Checks: Ensures uniform data formats, correct encoding, and no duplicate records.

Feature Engineering and Selection Tests

- Correlation Analysis: Uses Pearson/Spearman correlation to remove redundant features.
- Variance Inflation Factor (VIF): Detects multicollinearity in data.
- Principal Component Analysis (PCA): Reduces dimensionality while retaining meaningful information.
- Chi-Square Test: Determines if categorical features are statistically significant.

7.1.2 Model Evaluation Techniques

Performance Metrics for Classification Models

- Accuracy: Proportion of correctly classified instances out of total instances.
- Precision: Proportion of true positives among all predicted positives.
- Recall (Sensitivity): Proportion of true positives among all actual positives.

- F1-Score: Harmonic mean of precision and recall, balancing both metrics.
- Kappa Coefficient: Measures agreement between predicted and actual classes, accounting for chance.

Cross-Validation Techniques

- K-Fold Cross-Validation: Splits data into K subsets, training the model multiple times on different folds to ensure robust evaluation.
- Leave-One-Out Cross-Validation (LOOCV): Trains on all data except one observation, repeating for all observations.

7.1.3 Hyperparameter Tuning

- Grid Search: Systematically tests all combinations of specified parameter values.
- Random Search: Randomly selects combinations to find an optimal model faster.
- Bayesian Optimization: Uses probabilistic models to find the best parameters efficiently.

7.1.4 Performance and Stability Testing

Model Stability Testing

- Train-Test Split: Evaluates model performance on unseen data with an 80:20 ratio.
- Data Drift Detection: Identifies if new incoming data significantly deviates from training data.
- SHAP Analysis: Interprets feature importance in decision-making.

Bias and Fairness Testing

- Demographic Parity Test: Checks if the model predicts equally across student demographics (gender, qualification level).
- Equalized Odds Test: Ensures the model's false positive/negative rates are similar across different groups.

7.1.5 User Interface Testing

- System Usability Scale (SUS): Measures students' ease of use and satisfaction with the application.
- A/B Testing: Compares outcomes between groups receiving different interventions to measure improvement.
- Longitudinal Tracking: Cohort analysis tracks literacy skill development over multiple semesters.

7.2 TEST CASES

Detailed test cases were designed to cover data validation, model evaluation, and application functionality. These test cases ensure that the system is robust, reliable, and production ready.

7.2.1 Data Validation Test Cases

Table 7.1: Data Validation Test Cases

TC ID	Test Case Name	Description	Expected Outcome
1	Missing Data Handling	Check if missing values in numerical columns are filled using .fillna(df.mean()).	No missing values remain in numeric fields after preprocessing.
2	Duplicate Records Removal	Check if duplicate rows are present and removed from the dataset.	The dataset should not contain any identical rows.
3	Label Encoding Check	Verify that all categorical features (GENDER, EDUCATIONAL QUALIFICATION, LABEL) are encoded.	These columns should be transformed into numerical values using LabelEncoder().
4	Outlier Detection in Inputs	Identify abnormally high/low values in model input features.	Inputs outside expected range should be flagged or capped if needed.

7.2.2 Predictive Model Evaluation Test Cases

Table 7.2: Predictive Model Evaluation Test Cases

TC ID	Test Case Name	Description	Expected Outcome
1	Model Training Verification	Ensure all models (XGBoost, SVM, AdaBoost) are trained and can predict without errors.	All three models trained successfully with no runtime errors.
2	Input Shape Match	Verify input feature dictionary matches the training feature structure (X_train.columns).	The number of input features should exactly match training columns.
3	Prediction Output Format	Ensure the prediction output is a valid encoded label.	Prediction must be an integer/class value that exists in the LABEL encoder.
4	Algorithm Selection	Confirm the selected algorithm from UI (XGBoost, SVM, AdaBoost) returns correct model behavior.	The selected model should be used in prediction; not hardcoded.

CHAPTER – 8

RESULTS

This chapter presents the results obtained from the Information Literacy Prediction System. The screenshots demonstrate the complete workflow of the application, from dataset loading through student prediction, suggestion generation, and faculty resource management.

8.1 DATASET SCREENSHOT

The following screenshot shows the dataset features loaded from the Excel file (DATA.xlsx), displaying the behavioral attribute columns and the target LABEL column.

	T	U	V	W	X	Y	Z	AA	AB	AC	AD	AE	AF	AG
1	Collaborate using digi		Develop in	Promote I	Use learni	Maintain e	Promote r	Understan	Respect in	GENDER	EDUCATIO	LABEL		
2	97	92	88	99	96	89	96	88	84		P.hd	Excellent		
3	50	58	46	57	56	57	59	57	54	Male	UG	Average		
4	33	1	25	1	30	9	5	39	14	Male	PG	Poor		
5	97	98	94	100	81	91	100	91	96	Male	P.hd	Excellent		
6	67	74	62	62	74	76	74	66	65	Male	PG	Good		
7	68	75	68	69	65	61	80	66	68	Female	PG	Good		
8	83	94	100	90	95	96	98	95	92	Female	P.hd	Excellent		
9	1	30	25	35	40	9	8	18	26	Female	UG	Poor		
10	91	81	95	85	97	91	94	83	91		P.hd	Excellent		
11	48	56	56	58	44	43	57	54	52	Male	UG	Average		
12	93	87	84	98	97	91	84	82	82	Male	P.hd	Excellent		
13	92	95	94	90	86	97	92	87	93	Female	P.hd	Excellent		
14	91	87	82	88	93	100	98	86	93	Female	P.hd	Excellent		
15	32	12	31	39	39	38	19	8	34	Male	P.hd	Poor		
16	46	57	53	46	47	47	48	41	45	Female	UG	Average		
17	64	68	63	80	61	75	63	66	80	Male	PG	Good		
18	81	87	82	81	92	85	99	96	87	Female	P.hd	Excellent		
19	74	64	67	78	64	64	72	79	80	Female	PG	Good		

Fig. 8.1: Dataset Features

8.2 APPLICATION SCREENSHOTS

The following screenshots illustrate the complete user interface of the Information Literacy Prediction System, demonstrating each stage of the application workflow.

Home Page

The home page provides an overview of the system and navigation options for students and faculty to login or register.

Fig. 8.2: Home Page

Signup Page

New users both students and faculty can register by providing their name, username, password, email, gender, age, and role.

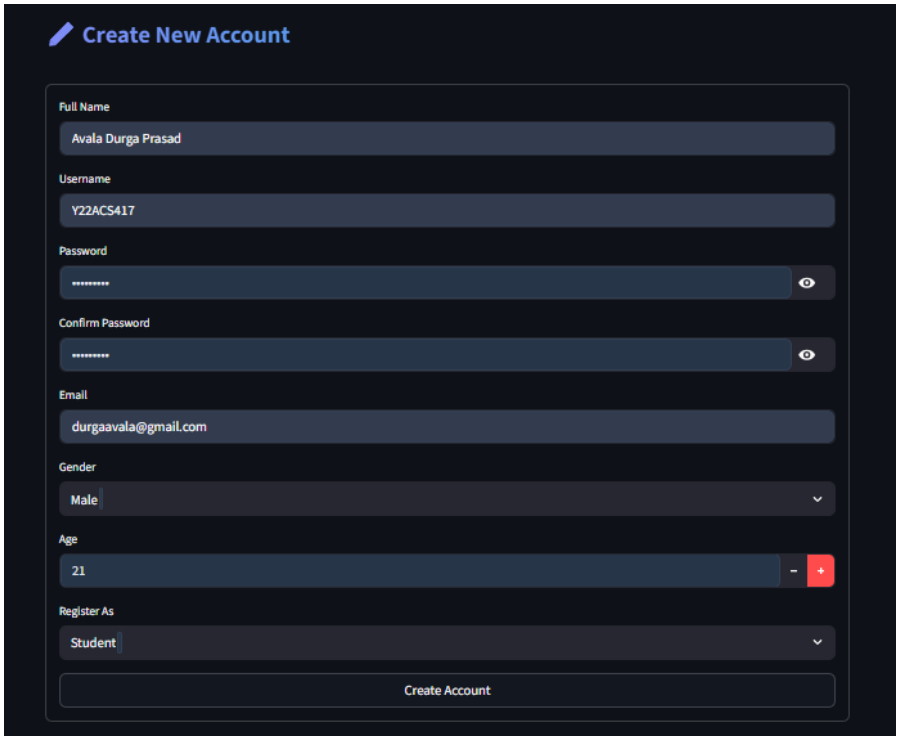

Fig. 8.3: Signup Page

Login Page

Registered users can log in using their username and password. The system validates credentials against the SQLite database and redirects to the appropriate dashboard based on the user's role.

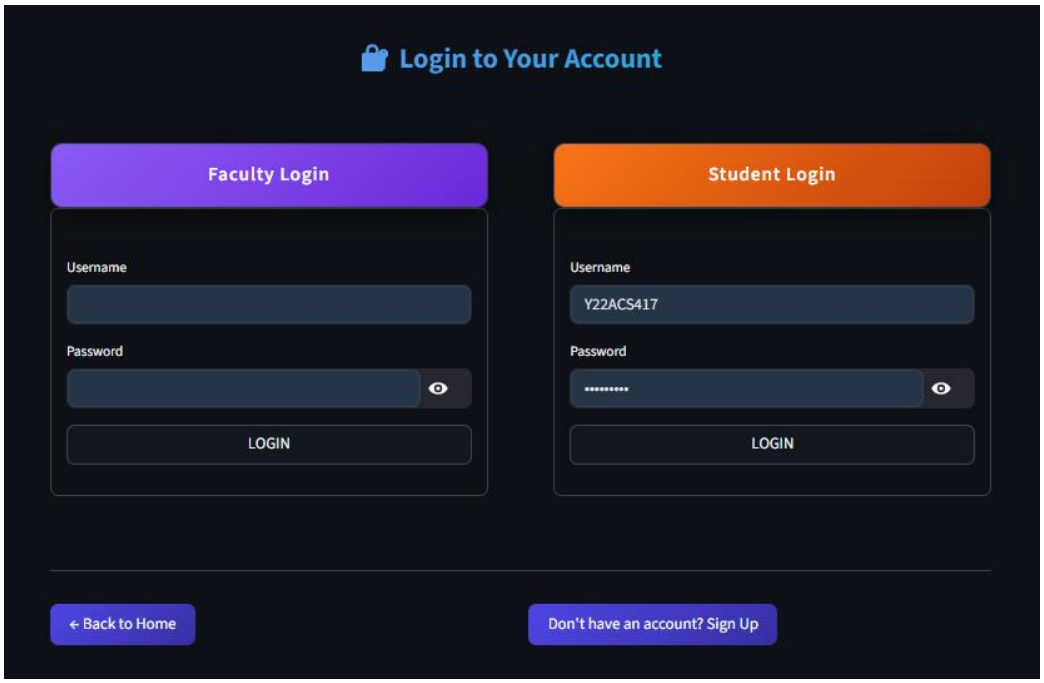

Fig. 8.4: Login Page

Student Feature Input

Students enter their behavioral scores across all information literacy dimensions. The input form is divided into multiple sections for ease of data entry.

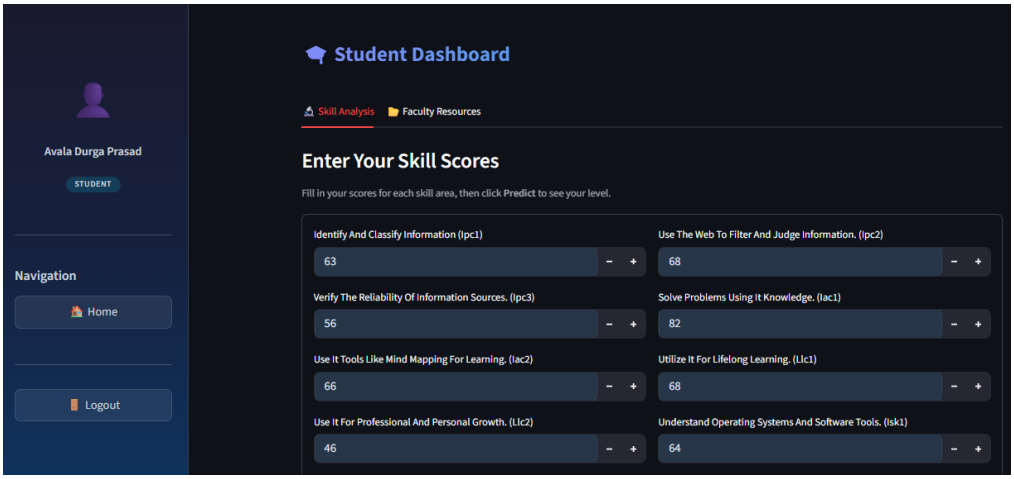

Fig. 8.5: Student Feature Input – Part 1

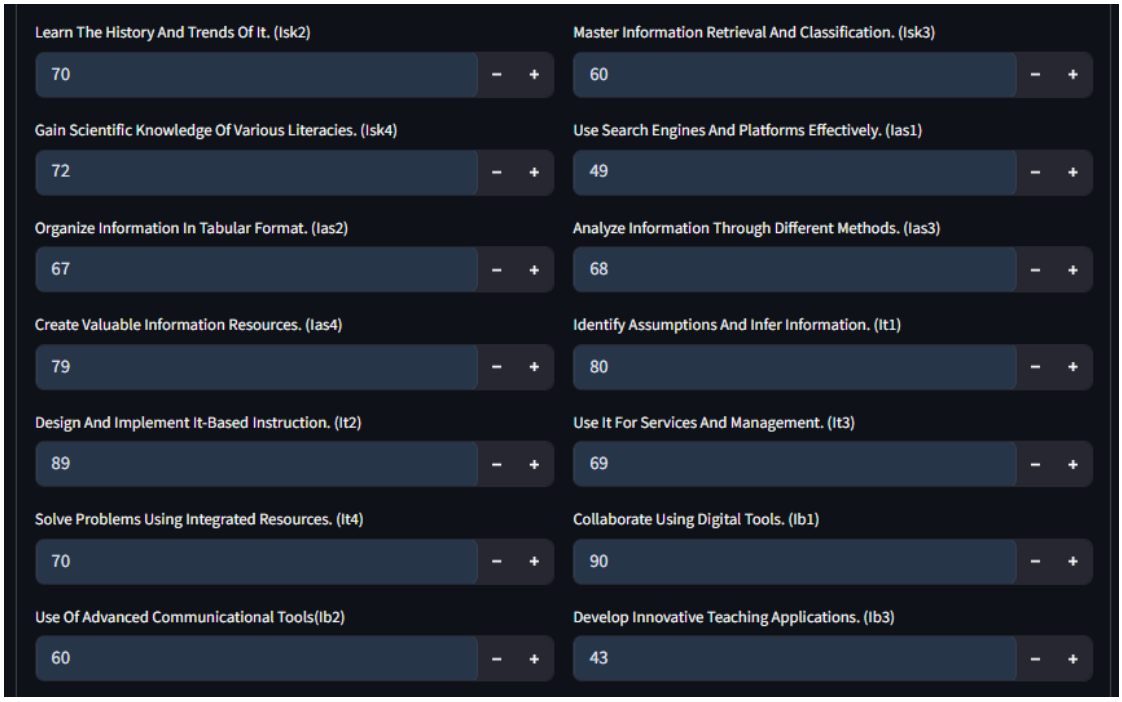

Fig. 8.6: Student Feature Input – Part 2

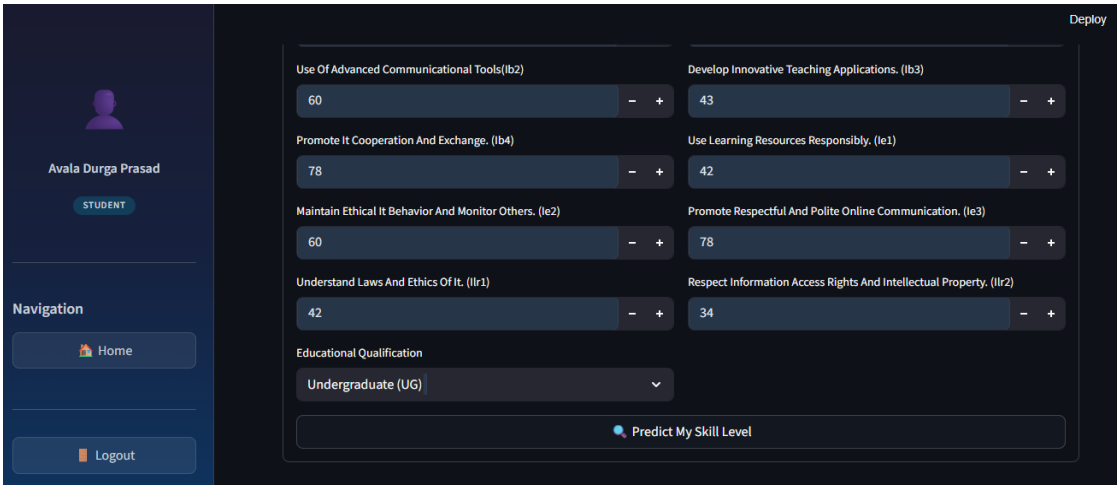

Fig. 8.7: Student Feature Input – Part 3

Prediction Result

After submitting the input data, the system applies the selected machine learning algorithm (XGBoost, SVM, or AdaBoost) and displays the predicted information literacy level along with model accuracy.

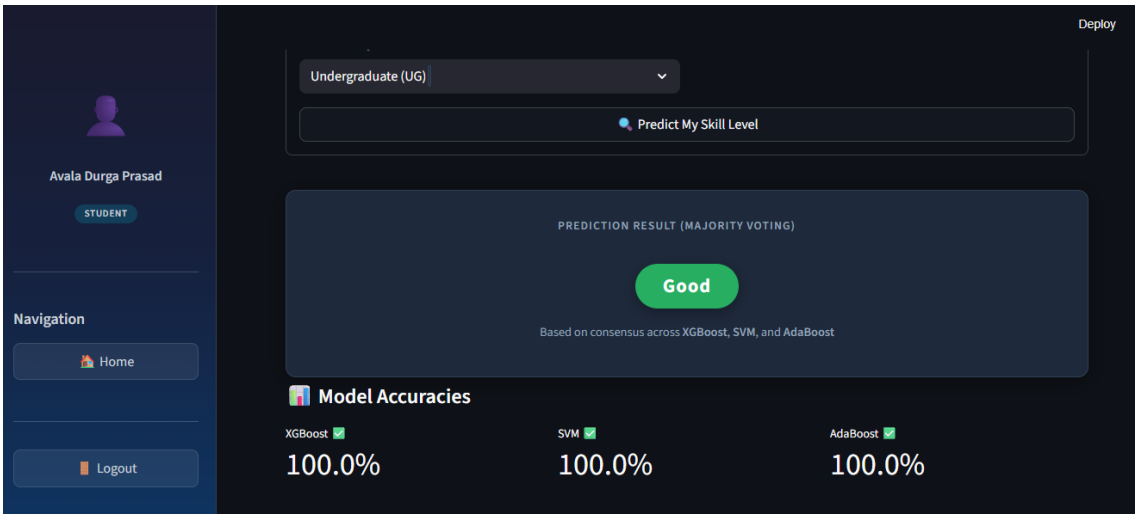

Fig. 8.8: Prediction Result

Personalized Suggestions

Based on the predicted literacy level, the system provides personalized suggestions to help the student improve their information literacy skills.

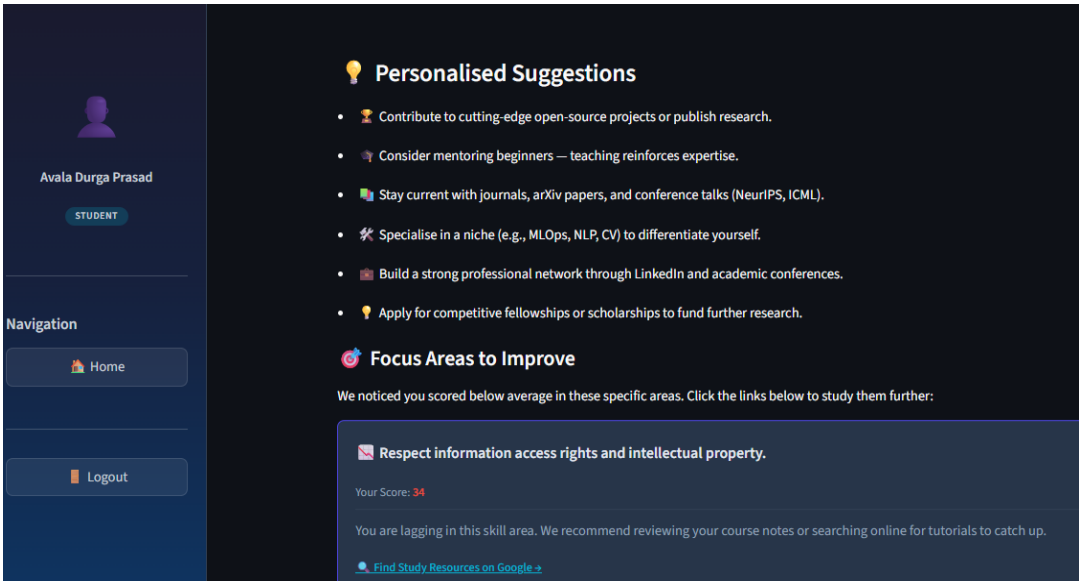

Fig. 8.9: Suggestions Based on Student Level

The system also identifies the specific weak skill areas of the student and provides targeted recommendations for improvement.

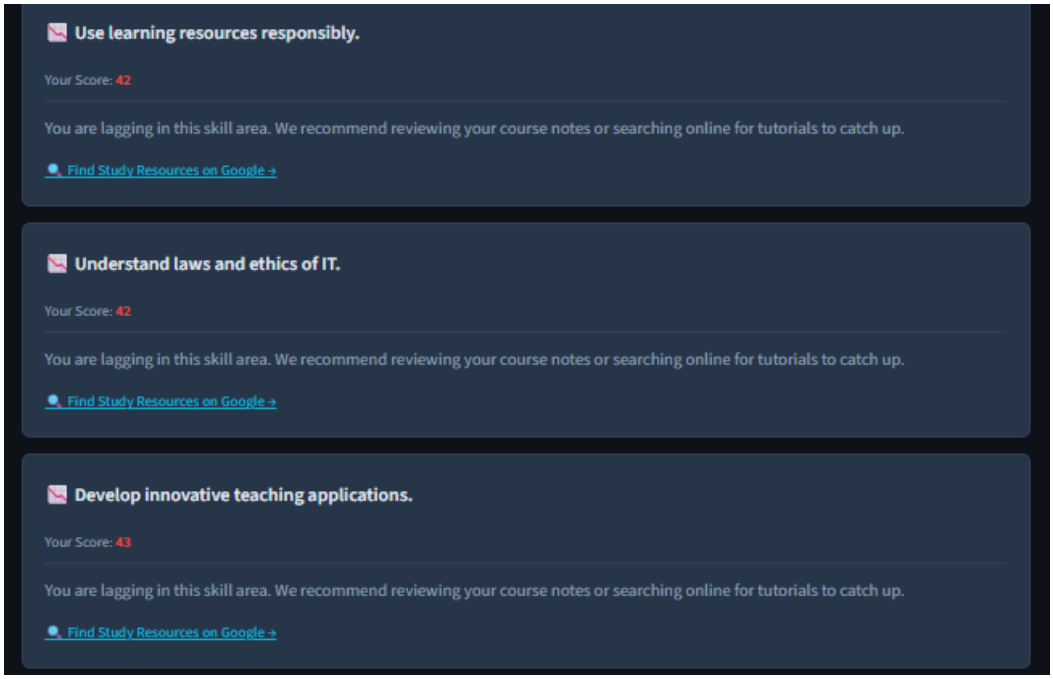

Fig. 8.10: Suggestions to Improve Weak Skill Areas

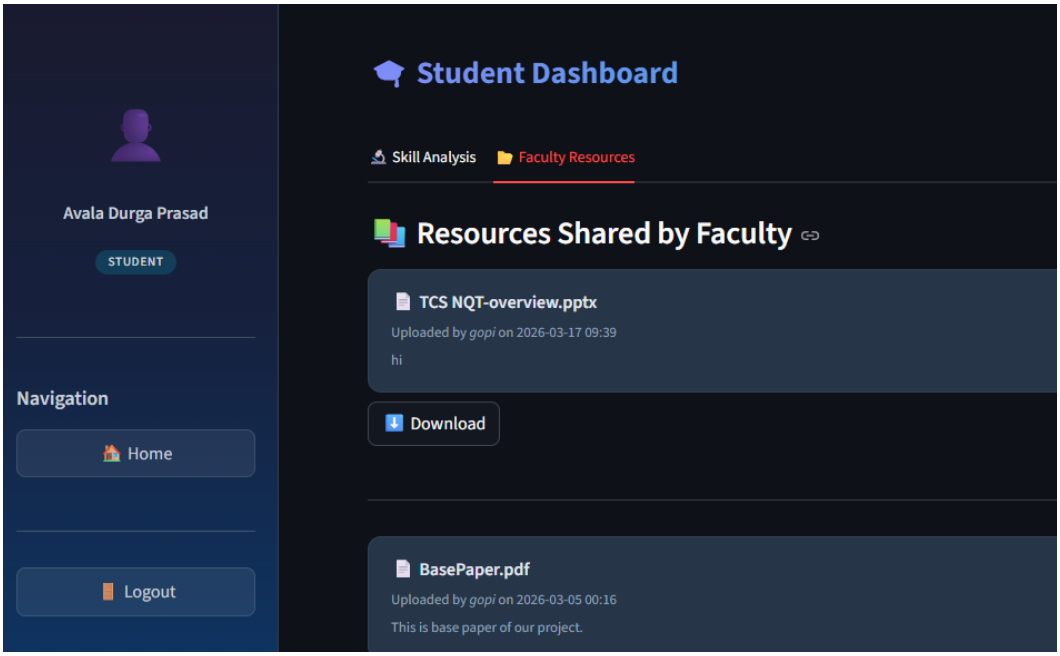

Fig. 8.11 – Student Dashboard Showing Faculty Shared Resources

Faculty Dashboard

The faculty dashboard enables educators to upload study materials and notes for students. Faculty can browse, upload, and manage resources from this interface.

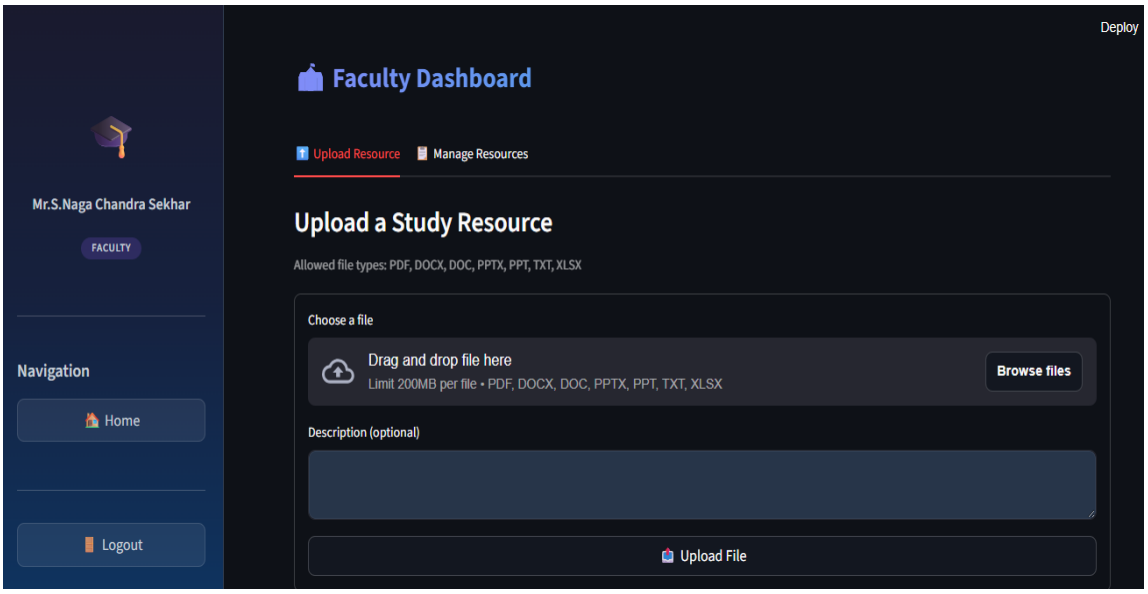

Fig. 8.12: Faculty Dashboard – Upload Resources

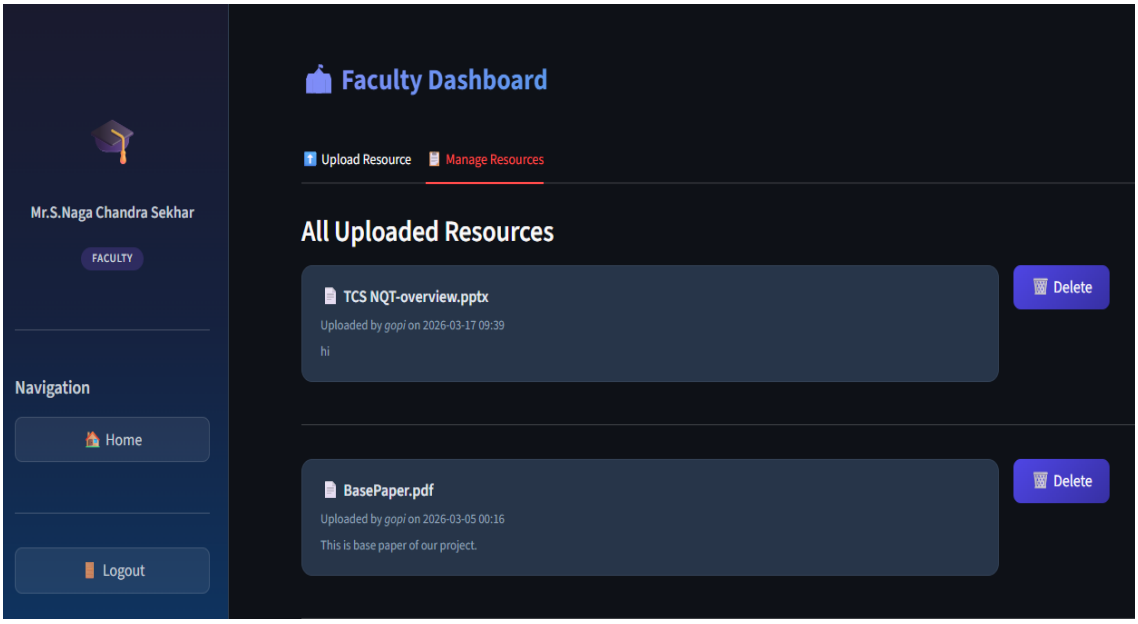

Fig. 8.13: Faculty Dashboard – Manage Resources

CONCLUSION

Predictive analytics in college information literacy is emerging as a powerful strategy to proactively identify at-risk students, tailor instructional methods, and improve academic achievement. Its true value lies not just in the data it collects, but in how effectively that data is tested, validated, and applied. Rigorous evaluation of data quality, model reliability, and intervention outcomes is essential to ensure accuracy and build trust in the system.

This project successfully implemented three advanced machine learning algorithms XGBoost, Support Vector Machine (SVM), and AdaBoost to predict students' information literacy levels with greater accuracy than traditional methods. By using ensemble and boosting techniques, the system reduces prediction bias and variance while maintaining computational efficiency. The application delivers a complete, end-to-end solution through a Streamlit web interface backed by a SQLite database.

In addition to prediction, this project enhances the learning process by identifying weak skill areas of individual students and providing personalized suggestions for improvement, enabling more effective and focused learning. A well-validated predictive analytics framework does more than enhance academic performance it transforms the student experience by equipping educators with actionable insights and students with timely support.

The inclusion of a faculty module for uploading study materials and notes allows students to access relevant resources continuously, supporting learning beyond prediction. Over time, these benefits contribute to improved retention rates, reduced academic disparities, and enhanced career preparedness. Ultimately, integrating predictive analytics with suggestion mechanisms and faculty support positions colleges to not only react to challenges but to anticipate and prevent them—laying the foundation for a more personalized, equitable, and future-ready education system.

FUTURE ENHANCEMENT

The current system provides a strong foundation for information literacy prediction and personalized learning support. Several enhancements can be explored in future work to further improve the system's capabilities, accuracy, and usability:

1. Advanced Machine Learning Models: Deep learning models such as LSTM, Transformer-based networks, or Graph Neural Networks can be explored to capture temporal and relational patterns in student behavior data, potentially improving prediction accuracy beyond the current ensemble approaches.
2. Real-Time Data Integration: The system can be enhanced to integrate real-time data streams from Learning Management Systems (LMS) such as Moodle or Canvas, enabling dynamic and continuous monitoring of student performance.
3. Enhanced Personalization: Natural Language Processing (NLP) can be applied to analyze student queries and learning notes, generating more granular and contextually relevant improvement suggestions.
4. Cloud Deployment: Migrating the application to cloud platforms such as AWS, Google Cloud, or Azure will improve scalability, availability, and accessibility for a larger number of concurrent users.
5. Integration with Online Learning Platforms: Connecting the system with platforms such as Google Classroom, Coursera, or edX will allow real-time tracking of course completion, quiz performance, and engagement metrics.
6. Smart Job Role Recommendation: Based on a student's information literacy profile and skill strengths, the system can recommend suitable career paths and learning tracks tailored to their academic and professional goals.
7. AI Chatbot Support: An AI-powered chatbot integrated into the platform can answer student queries in real time, clarify doubts, and provide guided learning support 24/7.
8. Explainability (XAI): Incorporating SHAP (SHapley Additive exPlanations) visualizations will help students and educators understand why a particular prediction was made, increasing trust and transparency in the system.

REFERENCES

Books

[1]. Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer Science & Business Media.

[2]. Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.). O'Reilly Media.

[3]. Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson Education.

[4]. Duda, R. O., Hart, P. E., & Stork, D. G. (2001). Pattern Classification (2nd ed.). Wiley-Interscience.

[5]. Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2016). Data Mining: Practical Machine Learning Tools and Techniques (4th ed.). Morgan Kaufmann.

[6]. Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction (2nd ed.). Springer.

Journal Papers

[1]. Shi, Y., Sun, F., Zuo, H., & Peng, F. (2023). Analysis of Learning Behavior Characteristics and Prediction of Learning Effect for Improving College Students' Information Literacy Based on Machine Learning. IEEE Access. https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10130155.

[2]. Guoxiang, Z., Elnura, M., & Guihua, W. (2023). Research on the Impact of Information Literacy on the Creativity of Foreign Language Teachers in Chinese Universities Under the Background of Big Data. International Journal of Distance EducationTechnologies,21(2). https://www.researchgate.net/publication/371233541.

[3]. Press, N., Arumugam, P. P., & Ashford-Rowe, K. (2019). Defining Digital Literacy: A Case Study of Australian Universities. ASCILITE 2019 Conference Proceedings, pp. 255–263. https://www.researchgate.net/publication/364627336

[4]. Mandušić, D., & Blašković, L. (2013). Information Literacy, Theory and Practice in Education. Revista Românească pentru Educaţie Multidimensională, 5(1), 47–58. https://www.researchgate.net/publication/285682216

[5]. Yan, D., & Li, G. (2023). A Heterogeneity Study on the Effect of Digital Education Technology on the Sustainability of Cognitive Ability for Middle School Students. Sustainability, 15, 2784. https://www.mdpi.com/2071-1050/15/3/2784

URLs

[1]. Scikit-Learn Documentation. Machine Learning in Python. https://scikit-learn.org/stable/

[2]. XGBoost Documentation. XGBoost Parameters and API Reference. https://xgboost.readthedocs.io/en/stable/

[3]. Streamlit Documentation. Build data applications in Python. https://docs.streamlit.io/

[4]. Python SQLite3 Documentation. DB-API 2.0 interface for SQLite databases. https://docs.python.org/3/library/sqlite3.html

[5]. Pandas Documentation. Python Data Analysis Library. https://pandas.pydata.org/docs/

[6]. Kaggle. Information Literacy Dataset and Machine Learning Notebooks. https://www.kaggle.com/

[7]. IEEE Xplore Digital Library. https://ieeexplore.ieee.org/